the root, in this case), we can fix the heap invariants with a right rotation; incidentally, we by doing so we will get a perfectly balanced tree.

We need to be clear now: by discarding the meaning of the priority field, we are implementing something different than what we had in section 3.3: in particular, this new data structure will no longer adhere to the priority queue public interface, it will not offer any `top` or `peek` method, instead it will just be a binary search tree, that `internally` uses the concepts we developed for treaps to maintain its structure balanced. Table 3.3 shows the methods and contract for BSTs: the data structure we are about to introduce will adhere to this API.

Table 3.3 API and contract for BinarySearchTree (BST)

Abstract Data Structure: Binary Search Tree	

API	`class BST {`
	`  insert(element)`
	`  remove(element)`
	`  contains(element)`
	`  min()`
	`  max()`
	`}`

Contract with client	Entries are kept sorted by element (*aka* key).

3.4.2 Introducing Randomization

If getting better results using simpler algorithms sounds too good to be true to you… well, you might be partially right, meaning that there is a price to pay for simplicity.

Updating priorities to maintain the tree balanced seemed easy on our small example, but doing it systematically on a large tree